

Chapitre 3 – Un peu d’interaction

Notes sur le chapitre

Le programme de la spécialité annonce dans son préambule que « les interfaces qui permettent la communication avec les humains, la collecte des données et la commande des systèmes » sont un « élément transversal » du programme. Cependant, les éléments du programme consacrés aux « interactions entre l’homme et la machine » concernent seulement l’interaction sur le Web. Ces éléments sont couverts dans le chapitre 11 de ce manuel.

Il nous a cependant semblé important d’introduire quelques éléments de programmation d’interfaces textuelles et graphiques qui ne fassent pas appel aux technologies du Web et qui puissent être utilisés aisément par les élèves dans leurs programmes Python. C’est pourquoi ce chapitre présente les principes de base de l’interaction textuelle et de l’interaction graphique.

L’**interaction textuelle** aborde l’affichage de texte, avec notamment les chaînes de formatage, et la saisie de données textuelles et numériques, ce qui permet d’introduire la gestion d’erreurs avec l’instruction `try`. L’utilisation de l’instruction `try` lorsque l’on essaie de lire une entrée numérique nous semble la meilleure solution à ce stade car Python offre peu d’alternatives sauf à utiliser des concepts avancés (expressions régulières) ou des bibliothèques spécialisées. Par exemple, une méthode telle que `s.isdigit()` permet de savoir si `s` est constitué uniquement de chiffres, mais n’est pas suffisante pour savoir si `s` représente un nombre flottant ou négatif : `'12.45'.isdigit()` retourne `False`, de même que `'-12'.isdigit()`.

L’**interaction graphique** utilise une bibliothèque spécialement développée pour ce manuel, `nsi_ui` («ui» signifie « user interface », c’est-à-dire « interface utilisateur »). Cette bibliothèque exporte un ensemble de fonctions permettant de créer et présenter les interacteurs de base. Elle fonctionne dans les environnements de programmation Python tels que EduPython grâce à la bibliothèque standard Python `tkinter`, et dans l’environnement Jupyter avec la bibliothèque d’interacteurs de Jupyter `ipywidgets`. Les interacteurs ont été choisis pour être comparable à ceux qui seront introduits au Chapitre 11 pour l’interaction avec les formulaires sur le Web.

Le principal écueil de ce chapitre est la notion de **fonction de rappel**, qui est la pierre angulaire de la programmation par événements qui caractérise les interfaces graphiques. Une fonction de rappel est une fonction dont le *nom* est passé en paramètre à une autre fonction, qui peut ainsi l’appeler lorsqu’elle en a besoin. On retrouvera ce principe au chapitre 6 avec la fonction `sort` de Python pour trier des tableaux, qui peut prendre en paramètre une fonction de comparaison d’éléments.

Par ailleurs la programmation par événements conduit souvent à utiliser des variables globales afin que les différentes fonctions de rappel puissent partager l’état du programme. Les variables globales sont considérées comme une mauvaise pratique en programmation, aussi faut-il essayer de limiter leur usage au strict minimum.

Note : La compatibilité de la bibliothèque `nsi_ui` avec la bibliothèque graphique Qt, annoncée page 51 du manuel, n’a finalement pas été jugée utile et n’est donc pas fournie.

Activité : Contrôler la tortue

Des boutons pour lancer des fonctions

```
def polygone(n, cote):  
    """ Trace un polygone de n cotes de taille `cote` """  
    for i in range(n):  
        forward(cote)  
        right(360/n)  
  
button("Effacer", clear)
```

```
main_loop()
```

Des tirettes pour contrôler les paramètres

```
tirette_taille = slider("Taille", 10, 100)
tirette_ncotes = slider("Nombre de cotes", 3, 50)

def creer_polygone():
    polygone(get_int(tirette_ncotes), get_int(tirette_taille))

button("Polygone", creer_polygone) # bouton pour faire le dessin
button("Effacer", clear)

main_loop()
```

Piloter la tortue au clavier

```
def gauche():
    left(10)

def droite():
    right(10)

onkey(droite, "Right")
onkey(gauche, "Left")

main_loop()
```

QCM (CHECKPOINT)

1 c ; 2 c ; 3 c ; 4 a, c et e ; 5 b ; 6 b et c ; 7 c ; 8 b.

Exercices

Exercices Newbie

1

```
def input_bool(message):
    """ Demande une saisie booléenne """
    while True:
        r = input(message)
        if r == 'o' or r == 'oui':
            return True
        if r == 'n' or r == 'non':
            return False
        print('Répondre oui ou non !')

input_bool('Fait-il beau ? ')
```

2 a.

```
temp = input('Température à convertir : ')
print(float(temp)*9/5 + 32)
```

b.

```
temp = input('Température à convertir : ')
unite = input('Centigrades (C) ou Fahrenheit (F) ? ')
if unite == 'C':
    print(float(temp)*9/5 + 32)
elif unite == 'F':
    print((float(temp) - 32) * 5/9)
else:
    print('Unité non reconnue')
```

3

```
entree = input('Entrer un chiffre entre 1 et 9 : ')
n = int(entree)
if n > 0 and n < 10:
    for i in range(1, 11):
        print(f'{i:2} x {n} = {i*n:2}')
else:
    print('Entrée invalide')
```

4

```
def input_default(message, default):
    """ Demande un saisie et retourne `default`
        si l'utilisateur tape simplement Entree
    """
    reponse = input(f'{message}({default}) ')
    if reponse == "":
        return default
    return reponse

input_default("Ville de départ ? ", "Paris")
```

5

```
from nsi_ui import *

# Note : nécessaire seulement dans Jupyter
clear_ui()

# Monnaies et taux de change
monnaie1 = "EUR" #input('Monnaie 1 ?')
monnaie2 = "USD" #input('Monnaie 2 ?')
taux = 1.20 #input('Taux de conversion ?')

def convert():
    """ Convertit le montant saisi dans les deux monnaies """
    m = get_float(montant)
    r1 = m * taux
    r2 = m / taux
    set_text(conv1, str(m) + str(monnaie1) + " = " + str(r1) + str(monnaie2))
    set_text(conv2, str(m) + str(monnaie2) + " = " + str(r2) + str(monnaie1))

# Construire l'interface
montant = entry("Montant")
button("Convertir", convert)
begin_vertical()
conv1 = label('conversion')
conv2 = label('conversion')
end_vertical()

main_loop()
```

6 a.

```
from nsi_ui import *

# Note : nécessaire seulement dans Jupyter
clear_ui()

# champs de saisie de texte pour entrer les valeurs de a et b
champA = entry("Valeur de a")
champB = entry("Valeur de b")

def ajouter():
    """ Afficher la somme des valeurs des champs A et B """
    a = get_float(champA) # valeur entrée dans le champ A
    b = get_float(champB) # valeur entrée dans le champ B
    set_text(resultat, a+b) # afficher le resultat

def soustraire():
    """ Afficher la difference des valeurs des champs """
    a = get_float(champA)
    b = get_float(champB)
    set_text(resultat, a-b)

button("Différence", soustraire)
button("Somme", ajouter) # bouton pour lancer le calcul
resultat = label("") # zone d'affichage du résultat

main_loop() # lancer l'interface
```

b.

```
# Note : nécessaire seulement dans Jupyter
clear_ui()

def ajouter():
    """ Afficher la somme des valeurs des champs A et B """
    a = get_float(champA) # valeur entrée dans le champ A
    b = get_float(champB) # valeur entrée dans le champ B
    set_text(resultat, f'Somme = {a+b}') # afficher le resultat

def soustraire():
    """ Afficher la difference des valeurs des champs """
    a = get_float(champA)
    b = get_float(champB)
    set_text(resultat, f'Différence = {a-b}')

# Construction de l'interface
begin_vertical()
begin_horizontal()
champA = slider("Valeur de a", 0, 100)
champB = slider("Valeur de b", 0, 100)
end_horizontal()

begin_horizontal()
button("Somme", ajouter)
button("Différence", soustraire)
end_horizontal()

resultat = label("")
end_vertical()

main_loop()
```

7

```

from nsi_ui import *

# Note : nécessaire seulement dans Jupyter
clear_ui()

# L'heure courante
H = 12
M = 0

def avance_heure():
    """ Ajouter 1 à l'heure H """
    global H
    H = H + 1
    if H > 23:
        H = 0
    set_value(heures, H)

def recule_heure():
    """ Retirer 1 de l'heure H """
    global H
    H = H - 1
    if H < 0:
        H = 23
    set_value(heures, H)

def avance_minute():
    """ Ajouter 1 aux minutes M """
    global M, H
    M = M + 1
    if M >= 59:
        M = 0
        avance_heure()
    set_value(minutes, M)

def recule_minute():
    """ Retirer 1 des minutes M """
    global M, H
    M -= 1
    if M < 0:
        M = 59
        recule_heure()
    set_value(minutes, M)

# Construire l'interface
button("-", recule_heure)
heures = entry("")
set_width(heures, 4)
button("+", avance_heure)
label("heure")

button("-", recule_minute)
minutes = entry("")
set_width(minutes, 4)
button("+", avance_minute)
label("minutes")

set_value(heures, H)
set_value(minutes, M)

main_loop()

```

8 a. La bibliothèque `ipyturtle` utilisée pour remplacer `turtle` dans Jupyter ne traite pas les événements d'entrée. Dans la version Jupyter de la correction, on remplace l'appui sur les touches du clavier par des boutons de la bibliothèque `nsi_ui`. Le fichier `exo8.py` contient le code du corrigé de cet exercice pour exécution en dehors de Jupyter.

```
from nsi_ui import *

def avance():
    forward(10)
def recule():
    backward(10)
def gauche():
    left(10)
def droite():
    right(10)

onkey(avance, 'Up')
onkey(recule, 'Down')
onkey(gauche, 'Left')
onkey(droite, 'Right')
```

b.

```
ecriture = True # etat du stylet

def stylet():
    """ Inverser l'état du stylet """
    global ecriture
    if ecriture:
        penup()
    else:
        pendown()
    ecriture = not ecriture

def pentagone():
    for i in range(5):
        forward(50)
        right(72)

onkey(stylet, '.')
onkey(pentagone, 'p')

mainloop()
```

Exercices Initié

9 a.

```
def input_int(message):
    """ Demande un nombre entier à l'utilisateur """
    while True:
        try:
            return int(input(message))
        except ValueError:
            print('erreur de saisie - recommencer')

input_int('Entrer un entier : ')
```

b.

```
def input_float(message):
    """ Demande un nombre décimal à l'utilisateur """
    while True:
        try:
            return float(input(message))
        except ValueError:
            print('erreur de saisie - recommencer')

input_float('Entrer un nombre décimal : ')
```

c.

```
def input_int_interval(message, min, max):
    """ Demande un nombre entier dans l'intervalle donné """
    v = input_int(message)
    while v < min or v > max:
        if v < min:
            print('Valeur trop petite')
        else:
            print('Valeur trop grande')
        v = input_int(message)
    return v

def input_float_interval(message, min, max):
    """ Demande un nombre décimal dans l'intervalle donné """
    v = input_float(message)
    while v < min or v > max:
        if v < min:
            print('Valeur trop petite')
        else:
            print('Valeur trop grande')
        v = input_float(message)
    return v

input_int_interval('Entree une valeur entre 0 et 100', 0, 100)
```

10 a.

```
from random import randint

secret = randint(0, 100) # nombre à trouver
trouve = False
while not trouve:
    n = input_int('Entrer votre : ') # exercice 9
    if n < secret:
        print(' trop petit')
    elif n > secret:
        print(' trop grand')
    else:
        print('Bravo !')
        trouve = True
```

b.

```
from random import randint

secret = randint(0, 100) # nombre à trouver
trouve = False
nessais = 0 # nombre d'essais
while not trouve:
    n = input_int('Entrer votre : ') # exercice 9
    nessais = nessais + 1
    if n < secret:
        print(' trop petit')
```

```

elif n > secret:
    print(' trop grand')
else:
    if nessais <= 6:
        print(f'Bravo ! trouvé en {nessais} essais')
    else:
        print(f'Trouvé en {nessais} essais')
    trouve = True

```

c.

```

from random import randint

secret = randint(0, 100) # nombre à trouver
trouve = False
nessais = 0 # nombre d'essais
minimum = 0 # valeur minimale de l'intervalle
maximum = 100 # valeur maximale de l'intervalle

while not trouve:
    n = input_int('Entrer votre : ') # exercice 9
    nessais = nessais + 1
    if n < secret:
        if n <= minimum:
            print(' je vous croyais plus malin !')
        else:
            print(' trop petit')
        minimum = n

    elif n > secret:
        if n >= maximum:
            print(' je vous croyais plus malin !')
        else:
            print(' trop grand')
        maximum = n

    else:
        if nessais <= 6:
            print(f'Bravo ! trouvé en {nessais} essais')
        else:
            print(f'Trouvé en {nessais} essais')
        trouve = True

```

11

```

from math import log10
n = input_int('entrer n : ') # exercice 9
n1 = 1 + int(log10(n))
n2 = 1 + int(log10(n*n))
for i in range(n):
    carre = (i+1)*(i+1)
    print(f'le carré de {i+1:{n1}} est {carre:{n2}}')

```

12 a.

```

insectes = 10
maximum = input_int("Nombre maximal d'insectes ? ") # exercice 9
njours = 1

while insectes * 3 < maximum:
    insectes = insectes * 3
    njours = njours + 1

print(f'{insectes} insectes en {njours} jours')

```


b.

```
from nsi_ui import *

# Utile seulement avec Jupyter
clear_ui()

def simulation():
    njours = 1
    # obtenir les paramètres
    facteur = get_int(entree_facteur)
    insectes = get_int(entree_initial)
    maximum = get_int(entree_maximum)
    # lancer la simulation
    while insectes * facteur < maximum:
        insectes = insectes * 3
        njours = njours + 1
    # afficher le résultat
    print(f'{insectes} insectes en {njours} jours')

entree_facteur = slider('Facteur de reproduction', 1, 10)
entree_initial = slider('Population initiale', 1, 100)
entree_maximum = slider('Population maximale', 1, 10000)
button('Simuler', simulation)

main_loop()
```

13

```
from nsi_turtle import * # Dans Python on peut importer turtle
from nsi_ui import *

# Utile seulement avec Jupyter
clear_ui()

def decompte():
    """ Appelé chaque seconde du décompte """
    compteur = get_int(compteur_texte)
    compteur = compteur - 1
    set_text(compteur_texte, compteur)
    if compteur <= 0:
        stop_animate()
        write("Decollage")
        left(90)
        forward(100)

def demarre():
    """ Lance le chronomètre """
    clear()
    compteur = get_int(compteur_texte)
    animate(decompte, 1000)

compteur_texte = entry("Compteur :")
button("Start", demarre)

main_loop()
```

14

```
from nsi_ui import *

# Utile seulement avec Jupyter
clear_ui()
```

```

# temps de début du dernier tour
# (0 ou bien dernier appui sur le bouton Lap)
dernier_tour = 0

def get_chrono():
    """ Retourne la valeur du chronomètre """
    return get_float(chrono_texte)

def compte():
    """ Mettre à jour le chronomètre """
    chrono = get_chrono()
    chrono += .1
    set_text(chrono_texte, round(chrono, 1))

def demarre():
    """ Lancer le chronomètre """
    global dernier_tour
    dernier_tour = 0
    set_text(chrono_texte, 0.0)
    set_text(tour_texte, 0.0)
    animate(compte, 100)

def arrete():
    """ Arrêter le chronomètre """
    stop_animate()

def tour():
    """ Afficher le temps au tour """
    global dernier_tour
    chrono = get_chrono()
    set_text(tour_texte, round(chrono - dernier_tour, 1))
    dernier_tour = chrono

# Créer l'interface
chrono_texte = entry("Chronometre : ")
set_width(chrono_texte, 12)
button("Start", demarre)
button("Lap", tour)
button("Stop", arrete)
tour_texte = entry("Dernier tour : ")
set_width(tour_texte, 12)

main_loop()

```

15 a.

```

from turtle import *

def affiche_nombre(heure):
    write(heure)

from nsi_ui import *

def dessine_cadran():
    """ Dessine le cadran de l'horloge analogique """
    penup()
    goto(0, 0)
    setheading(90)
    # affiche les 12 nombres des heures
    for i in range(1, 13):
        right(30)
        forward(50)
        affiche_nombre(i)
        backward(50)

```

```
goto(0,0)

dessine_cadran()
```

b.

```
def dessine_aiguille(angle, length):
    """ Affiche une aiguille à l'angle donné
        et de la longueur donnée
    """
    penup()
    goto(0,0)
    setheading(90)
    pendown()
    right(angle)
    forward(length)

def affiche_heure(H, M):
    """ Affiche les aiguilles correspondant à l'heure H:M """
    pensize(3)
    dessine_aiguille((H % 12) * 30, 20)
    pensize(1)
    dessine_aiguille(M * 6, 40)

def efface_heure(H, M):
    """ Affiche les aiguilles correspondant à l'heure H:M """
    pencolor("white")
    pensize(10)
    dessine_aiguille((H % 12) * 30, 20)
    dessine_aiguille(M * 6, 40)
    pensize(1)
    pencolor("black")

affiche_heure(10, 10)

efface_heure(10, 10)
affiche_heure(16, 40)
```

c.

```
# Note : nécessaire seulement dans Jupyter
clear_ui()

# L'heure courante
H = 16
M = 40

# On reprend l'interface de l'exercice 8 en ajoutant
# les appels à efface_heure et affiche_heure pour
# mettre à jour l'horloge analogique
#
def avance_heure():
    """ Ajouter 1 à l'heure H """
    global H, M
    efface_heure(H, M)

    H = H + 1
    if H > 23:
        H = 0
    set_value(heures, H)
    affiche_heure(H, M)

def recule_heure():
    """ Retirer 1 de l'heure H """
```

```

global H, M
efface_heure(H, M)

H = H - 1
if H < 0:
    H = 23
set_value(heures, H)
affiche_heure(H, M)

def avance_minute():
    """ Ajouter 1 aux minutes M """
    global H, M
    efface_heure(H, M)

    M = M + 1
    if M >= 59:
        M = 0
        avance_heure()
    set_value(minutes, M)
    affiche_heure(H, M)

def recule_minute():
    """ Retirer 1 des minutes M """
    global H, M
    efface_heure(H, M)

    M -= 1
    if M < 0:
        M = 59
        recule_heure()
    set_value(minutes, M)
    affiche_heure(H, M)

# Construire l'interface
button("-", recule_heure)
heures = entry("")
set_width(heures, 4)
button("+", avance_heure)
label("heure")

button("-", recule_minute)
minutes = entry("")
set_width(minutes, 4)
button("+", avance_minute)
label("minutes")

set_value(heures, H)
set_value(minutes, M)

main_loop()

```

16

```

from nsi_ui import *

# Note : nécessaire seulement dans Jupyter
clear_ui()

def FtoC():
    """ Convertit la température du slider en degrés centigrades
        et affiche le résultat
    """
    t = get_int(temperature)
    set_text(result, (t - 32.) * 5/9)

```

```
def CtoF():
    """ Convertit la température du slider en degrés Fahrenheit
        et affiche le résultat
    """
    t = get_int(temperature)
    set_text(result, t * 9/5 + 32.)

# Construire l'interface
temperature = slider("Temperature", -100, 100)
convert = button("C to F", CtoF)
convert = button("F to C", FtoC)
result = entry("Resultat")
set_width(result, 10)

main_loop()
```

17

```
from nsi_ui import *

# Utile seulement avec Jupyter
clear_ui()

def convertirF(v):
    # convertir de centigrade -> Fahrenheit
    set_value(fahrenheit, v * 9/5 + 32.)

def convertirC(v):
    # convertir de Fahrenheit -> centigrade
    set_value(celsius, (v - 32.) * 5/9)

# Créer l'interface
celsius = slider("centigrade", -100, 100, convertirF)
fahrenheit = slider("Fahrenheit", -148, 212, convertirC)
main_loop()
```

18

```
from turtle import *

from nsi_ui import *

def afficher():
    """ Reafficher le texte avec les attributs courants """
    clear()
    write(get_string(text))

def aligne(a):
    """ Changer l'alignement du texte """
    set_align(a)
    afficher()

def taille(t):
    """ Changer la taille du texte """
    set_size(t)
    afficher()

def style(s):
    """ Changer le style d'affichage """
    set_style(s)
    afficher()

# Construire l'interface
```

```

# entrée de texte
text = entry("Texte : ")
set_font('Helvetica')

# boutons pour l'alignement
begin_vertical()
button("Gauche", aligne, 'left')
button("Centre", aligne, 'center')
button("Droite", aligne, 'right')
end_vertical()

# tirette pour la taille
begin_vertical()
taille = slider("Taille", 10, 50, taille)

# boutons pour le style
begin_horizontal()
button("Normal", style, 'normal')
button("Gras", style, 'bold')
button("Italique", style, 'italic')
end_horizontal()

end_vertical()

main_loop()

```

Exercices Titan

19 a. Une première stratégie pourrait être de proposer des nombres aléatoires entre 0 et 100. Mais cela ne prend pas en compte l'information fournie par l'utilisateur.

Une meilleure stratégie est de garder en mémoire le plus petit nombre pour lequel l'utilisateur a indiqué que le nombre cherché est plus grand que lui (**minimum**), et le plus grand nombre pour lequel l'utilisateur a indiqué que la nombre cherché est plus petit que lui (**maximum**). On réduit ainsi l'intervalle à chaque tour.

Ici, on choisit un nombre au hasard entre le minimum et le maximum.

```

from random import randint

minimum = 0 # le nombre le plus petit pour l'instant
maximum = 100 # le nombre le plus grand pour l'instant
trouve = False # True lorsque l'on a trouvé le nom
nessais = 0 # nombre de coups

def choix(min, max):
    return randint(min+1, max-1)

print('Choisissez un nombre dans votre tête.')
print('À chacune de mes propositions, répondez')
print(' < si votre nombre est plus petit que celui que je propose')
print(' > si votre nombre est plus grand que celui que je propose')
print(' = si j'ai deviné le nombre')

while not trouve:
    essai = choix(minimum, maximum)
    nessais = nessais + 1
    reponse_ok = False
    while not reponse_ok:
        reponse = input(f'{essai} ? (répondre <, > ou =) ')

```

```

reponse_ok = True
if reponse == '>':
    minimum = essai
elif reponse == '<':
    maximum = essai
elif reponse == '=':
    trouve = True
    print(f"J'ai trouvé en {nessais} coups !")
else:
    reponse_ok = False
    print('réponse incorrecte')

```

Note : Une meilleure stratégie est de prendre le milieu de l'intervalle (il s'agit alors de la *recherche dichotomique* qui sera vue au chapitre 6) :

```

def choix(min, max):
    return (min + max) // 2

```

b. Le joueur triche s'il répond `<` ou `=` pour un nombre inférieur ou égal au minimum, ou `>` ou `=` pour un nombre supérieur au maximum.

Avec le code ci-dessus ce cas ne peut pas se produire puisque l'on choisit un nombre entre `minimum+1` et `maximum-1`. Cependant, si cet intervalle est vide, cela signifie que l'utilisateur a triché. Pour détecter ce cas il suffit d'ajouter un test à la fin de la boucle `while` :

```

if minimum == maximum or minimum+1 == maximum:
    print('Vous avez triché !')
    trouve = True

```

20 a.

```

from nsi_curses import *

from random import randint

minimum = 0 # le nombre le plus petit pour l'instant
maximum = 100 # le nombre le plus grand pour l'instant
trouve = False # True lorsque l'on a trouvé le nom
nessais = 0 # nombre de coups

def choix(min, max):
    return randint(min+1, max-1)

init_curses()
add_str(20, 10, 'Choisissez un nombre dans votre tête.')
add_str(20, 11, 'À chacune de mes propositions, répondre')
add_str(20, 12, ' < si votre nombre est plus petit que celui que je propose')
add_str(20, 13, ' > si votre nombre est plus grand que celui que je propose')
add_str(20, 14, ' = si j'ai deviné le nombre')
add_str(20, 15, "Taper une touche pour commencer")
# donner le temps au joueur de lire les règles et de choisir un nombre
c = wait_key()

while not trouve:
    # Note : la fonction clear() de nsi_curses efface l'écran
    clear()

    essai = choix(minimum, maximum)
    add_str(40, 20, f'{essai}')
    nessais = nessais + 1

    reponse_ok = False
    while not reponse_ok:

```

```

reponse = wait_key() # attente d'une touche
reponse_ok = True
if reponse == '>':
    minimum = essai
elif reponse == '<':
    maximum = essai
elif reponse == '=' || reponse == ' ':
    # On ajoute la barre d'espace pour faciliter le jeu
    # car le caractère '=' est difficile d'accès
    trouve = True
    clear()
    add_str(30, 20, f"J'ai trouvé en {nessais} coups !")
else:
    reponse_ok = False
    clear()
    add_str(30, 20, 'réponse incorrecte')

if minimum == maximum or minimum+1 == maximum:
    clear()
    add_str(30, 20, 'Vous avez triché !')
    trouve = True

add_str(30, 21, 'Taper une touche pour quitter')
c = wait_key()
end_curses()

```

b. Pour gérer le fait que `get_key` retourne une chaîne vide lorsque l'on n'a rien saisi dans l'intervalle donné, on ajoute un cas dans le corps de la boucle while :

```

while not reponse_ok:
    reponse = get_key(10) # attente au plus 1 seconde
    reponse_ok = True
    if reponse == '': # pas d'entrée
        trouve = True
        clear()
        add_str(30, 20, f'Trop lent ! vous avez perdu')
    elif reponse == '>':
        ...

```

21 a.

```

from nsi_ui import *

# Utile seulement avec Jupyter
clear_ui()

def rien():
    """ Fonction de rappel qui ne fait rien """
    pass

# Construire l'interface
begin_vertical()

# La zone résultat
res = entry("")
set_width(res, 16)

# La rangée du haut
begin_horizontal()
button('7', rien)
button('8', rien)
button('9', rien)
button('/', rien)
end_horizontal()

```



```
# La deuxième rangée
```

```
begin_horizontal()
button('4', rien)
button('5', rien)
button('6', rien)
button('*', rien)
end_horizontal()
```

```
# La troisième rangée
```

```
begin_horizontal()
button('1', rien)
button('2', rien)
button('3', rien)
button('+', rien)
end_horizontal()
```

```
# La rangée du bas
```

```
begin_horizontal()
button('C', rien)
button('0', rien)
button('=', rien)
button('-', rien)
end_horizontal()
```

```
end_vertical()
```

```
main_loop()
```

b.

```
from nsi_ui import *
```

```
# Utile seulement avec Jupyter
```

```
clear_ui()
```

```
# Le nombre en train d'être entré
```

```
a = 0
```

```
def chiffre(d):
```

```
    """ Entrer le chiffre d """
```

```
    global a
```

```
    # mettre à jour et afficher le nouveau nombre
```

```
    a = a*10 + d
```

```
    set_text(res, a)
```

```
def effacer():
```

```
    """ Remettre à zéro la calculette """
```

```
    global a
```

```
    a = 0
```

```
    set_text(res, a)
```

```
# Construire l'interface
```

```
# Pour chaque chiffre, on utilise la fonction de rappel
```

```
# `chiffre` en lui passant le chiffre correspondant à la touche
```

```
begin_vertical()
```

```
# La zone résultat
```

```
res = entry("")
```

```
set_width(res, 16)
```

```
# La rangée du haut
```

```

begin_horizontal()
button('7', chiffre, 7)
button('8', chiffre, 8)
button('9', chiffre, 9)
button('/', rien)
end_horizontal()

# La deuxième rangée
begin_horizontal()
button('4', chiffre, 4)
button('5', chiffre, 5)
button('6', chiffre, 6)
button('*', rien)
end_horizontal()

# La troisième rangée
begin_horizontal()
button('1', chiffre, 1)
button('2', chiffre, 2)
button('3', chiffre, 3)
button('+', rien)
end_horizontal()

# La rangée du bas
begin_horizontal()
button('C', effacer)
button('0', chiffre, 0)
button('=', rien)
button('-', rien)
end_horizontal()

end_vertical()

main_loop()

```

c. Note : la logique des fonctions `chiffre` et `operation` est ce qui fait la difficulté de cet exercice. Il faut en effet mémoriser la saisie du premier opérande et de l'opération, et appliquer l'opération une fois la fin du second opérande saisie, c'est-à-dire lorsque l'on tape soit `=`, soit une autre opération. On utilise deux variables globales pour mémoriser le premier opérande (`nb_en_attente`) et l'opération (`op_en_attente`), ainsi qu'un booléen (`debut_nombre`) qui signale le début de la saisie d'un nouveau nombre.

```

from nsi_ui import *

# Utile seulement avec Jupyter
clear_ui()

# Le nombre en cours de saisie
a = 0
# True si on est prêt à saisir un nouveau nombre
debut_nombre = True
# Le nombre déjà saisi, en attente d'une opération
nb_en_attente = 0
# L'opération en attente de saisie du second nombre
op_en_attente = ""

def chiffre(d):
    """ Entrer le chiffre d """
    global a, nb_en_attente, debut_nombre
    if debut_nombre:
        # Si on vient de lancer la calculatrice ou
        # si on vient de taper une opération,
        # on est au début d'un nombre :

```

```

    # on mémorise le nombre courant,
    # et on en commence un nouveau
    nb_en_attente = a
    a = d
    debut_nombre = False
else:
    # On est en cours de saisie d'un nombre :
    # ajouter le chiffre d à sa droite
    a = a*10 + d
    set_text(res, a)

def effacer():
    """ Remettre à zéro la calculatrice """
    global a, debut_nombre, nb_en_attente, op_en_attente
    a = 0
    debut_nombre = True
    nb_en_attente = 0
    op_en_attente = ""
    set_text(res, a)

def operation(op):
    """ Mémoriser l'opération op et exécuter l'opération
    en attente, s'il y en a une.
    """
    global a, debut_nombre, nb_en_attente, op_en_attente
    # Le résultat de l'opération est mis dans nb_en_attente
    # de manière à pouvoir enchaîner les opérations 8 + 10 - 2
    div_zero = False
    if op_en_attente == '+':
        a = nb_en_attente + a
    elif op_en_attente == '-':
        a = nb_en_attente - a
    elif op_en_attente == '*':
        a = nb_en_attente * a
    elif op_en_attente == '/':
        if a == 0:
            nb_en_attente = 0
            div_zero = True
        else:
            a = nb_en_attente / a
    # Note : il n'y a rien à faire si l'opération en attente est =

    # préparer la prochaine saisie
    debut_nombre = True
    op_en_attente = op

    # afficher le résultat
    if div_zero:
        set_text(res, "Division par 0 !")
    else:
        set_text(res, a)

# Construire l'interface
# Pour chaque chiffre, on utilise la fonction de rappel
# `chiffre` en lui passant le chiffre correspondant à la touche
# Pour chaque opération, on utilise la fonction de rappel
# `operation` en lui passant l'opération correspondant à la touche

begin_vertical()

# La zone résultat
res = entry("")
set_width(res, 16)

```

```

# La rangée du haut
begin_horizontal()
button('7', chiffre, 7)
button('8', chiffre, 8)
button('9', chiffre, 9)
button('/', operation, '/')
end_horizontal()

# La deuxième rangée
begin_horizontal()
button('4', chiffre, 4)
button('5', chiffre, 5)
button('6', chiffre, 6)
button('*', operation, '*')
end_horizontal()

# La troisième rangée
begin_horizontal()
button('1', chiffre, 1)
button('2', chiffre, 2)
button('3', chiffre, 3)
button('+', operation, '+')
end_horizontal()

# La rangée du bas
begin_horizontal()
button('C', effacer)
button('0', chiffre, 0)
button('=', operation, '=')
button('-', operation, '-')
end_horizontal()

end_vertical()

main_loop()

```

Bonus : En remplaçant les appels à `set_text(res, a)` ci-dessus par l'appel à la fonction `affichage(res)` ci-dessous, on obtient un affichage de l'opération en cours, au lieu d'afficher seulement le nombre en cours de saisie.

```

def affichage(res):
    """ Affiche l'opération en cours dans res """
    if op_en_attente == "" or op_en_attente == "=":
        set_text(res, a)
    else:
        if debut_nombre:
            set_text(res, f'{a} {op_en_attente}')
        else:
            set_text(res, f'{nb_en_attente} {op_en_attente} {a}')

```

22 a.

```

from turtle import *
from nsi_ui import *

def avance():
    forward(2)

animate(avance)

onkey(stop_animate, '.')

```

b.

```

def avance():
    forward(2)

def gauche():
    left(10)

def droite():
    right(10)

bouge = False
def start_stop():
    global bouge
    if bouge:
        stop_animate()
    else:
        animate(avance)
    bouge = not bouge

onkey(gauche, 'Left')
onkey(droite, 'Right')
onkey(start_stop, 'Space')

left(90)
main_loop()

```

c.

```

vitesse = 2 # vitesse de la tortue
def avance():
    forward(vitesse)

def gauche():
    left(10)

def droite():
    right(10)

def accelere():
    global vitesse
    vitesse = vitesse + 1

def ralentit():
    global vitesse
    # Note : on pourrait autoriser les vitesses négatives,
    # ce qui aurait pour effet de faire reculer la tortue
    if vitesse > 1:
        vitesse = vitesse -1

bouge = False
def start_stop():
    global bouge
    if bouge:
        stop_animate()
    else:
        # Note : on pourrait remettre la vitesse
        # à sa valeur par défaut
        animate(avance)
    bouge = not bouge

onkey(gauche, 'Left')
onkey(droite, 'Right')
onkey(accelere, 'Up')
onkey(ralentit, 'Down')

```

```
onkey(start_stop, 'Space')

left(90)
main_loop()
```

d. On prend le même programme en remplaçant la fonction `avance` par celle-ci :

```
def avance():
    # Note : on teste si on est sorti après avoir bougé la tortue
    # Ainsi, une fois la tortue sortie, on peut la faire tourner,
    # puis la faire redémarrer pour qu'elle rentre dans la zone.
    forward(vitesse)
    # On utilise des bornes plus petites que dans l'énoncé
    # (200 au lieu de 400) pour tester plus facilement
    borne = 200
    if abs(xcor()) > borne or abs(ycor()) > borne:
        print('Stopped !')
        start_stop()
```

e. On prend le même programme en remplaçant la fonction `avance` par celle-ci :

```
def avance():
    # Note : on teste si on est sorti après avoir bougé la tortue
    # Ainsi, une fois la tortue sortie, on peut la faire tourner,
    # puis la faire redémarrer pour qu'elle rentre dans la zone.
    forward(vitesse)
    # On utilise des bornes plus petites que dans l'énoncé
    # (100 au lieu de 400) pour tester plus facilement
    borne = 100
    if abs(xcor()) > borne:
        setheading(180 - heading())
        forward(vitesse)

    if abs(ycor()) > borne:
        setheading(360 - heading())
        forward(vitesse)
```

23 a. Note : Le code de cet exercice ne peut être exécuté dans Jupyter car `ipyturtle` ne reconnaît pas les événements d'entrée. Le corrigé est disponible dans le fichier `exo23.py` pour exécution directe dans Python.

```
# importer nsi_turtle de préférence à turtle
from nsi_turtle import *

# S'assurer que les déplacements de la tortue sont instantanés
speed(0)

def bouger_tortue(x, y):
    """ Déplacer la tortue à la position x, y """
    penup() # pour ne pas laisser de trace
    goto(x, y)
    pendown()

# fonction de rappel lorsque l'on clique dans la fenêtre de la tortue
onscreenclick(bouger_tortue)

main_loop()
```

b.

```
# importer nsi_turtle de préférence à turtle
from nsi_turtle import *
```

```
# fonction de rappel lorsque l'on déplace la tortue avec la souris  
ondrag(goto)  
  
main_loop()
```